

C# Fundamentos

Aula 11 — Variáveis, tipos, operadores e estruturas condicionais

Introdução

Aula focada nos conceitos essenciais de *C#*: como declarar e usar variáveis, entender tipos, aplicar operadores e controlar fluxo com condicionais.

Objetivo: fornecer bases práticas para escrever e interpretar trechos de código simples em **C#**.

01
Variáveis



Declaração e Inicialização

Em *C#*, declare com o tipo seguido do nome: int contador; e inicialize: int contador = 0;.

Use inicialização ao declarar para evitar valores indefinidos e facilitar leitura do código.



Escopo e Vida Útil

Variáveis locais existem dentro de blocos { }. Variáveis de classe persistem enquanto o objeto existir.

Escolha escopo mínimo necessário para reduzir erros e melhorar manutenção.

Convenções de Nome

Use *camelCase* para variáveis locais e *PascalCase* para propriedades públicas.

Nomes devem ser descritivos e curtos, evitando abreviações obscuras.



02
Tipos de Dados

Tipos Primitivos

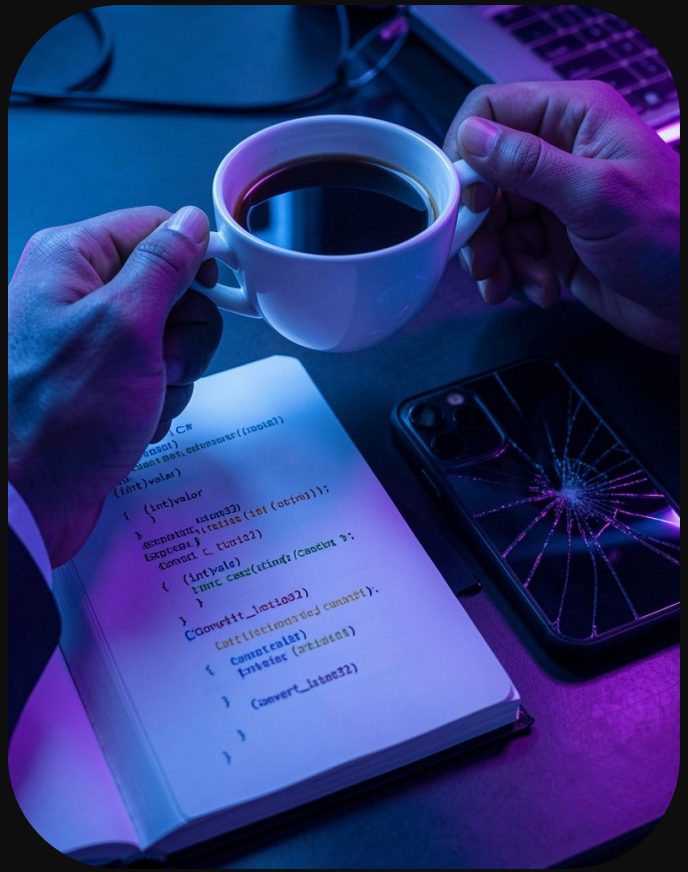
Tipos como *int, double, bool, char* são simples e têm tamanho fixo.

Escolha o tipo pelo domínio dos valores para otimizar memória e precisão.

Tipos Referência

Classes, arrays e *string* são referências; armazenam endereço, não cópia dos dados.

Atribuições compartilham referência — cuidado com mutabilidade e efeitos colaterais.



Conversão e Cast

Use conversão explícita com (Tipo)valor para *cast* e Convert/Parse para strings.

\n\nCuidado com perda de precisão ao converter de *double* para *int* e com exceções em conversões inválidas.



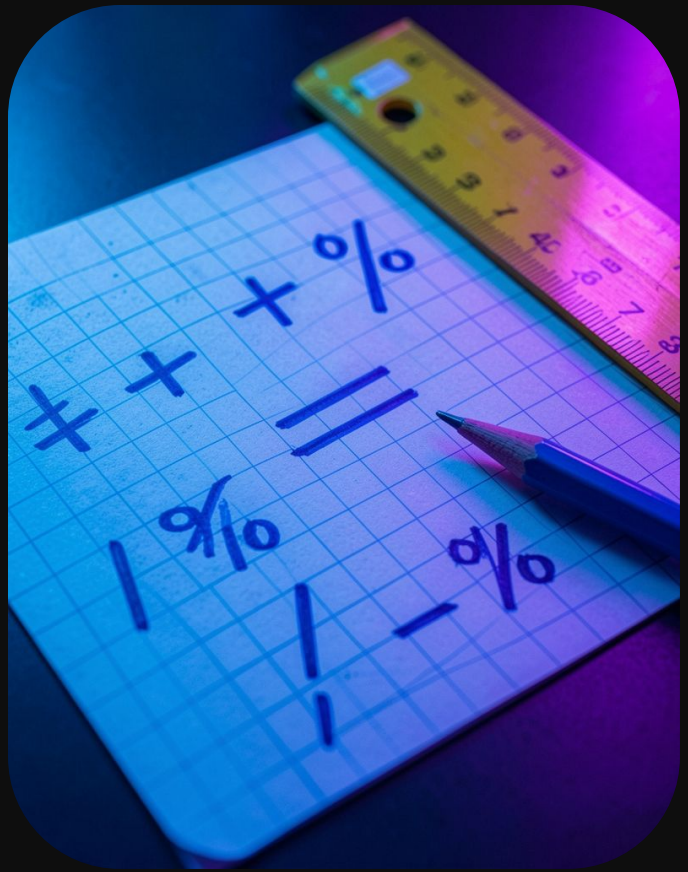
03

Operadores

Aritméticos e Atribuição

Operadores: + - * / %. Atribuição: =.

\n\nUse operadores aritméticos conforme tipo (inteiro vs ponto flutuante) para evitar truncamento.



Relacionais e Lógicos

Relacionais: == != = =. Lógicos: && || !.

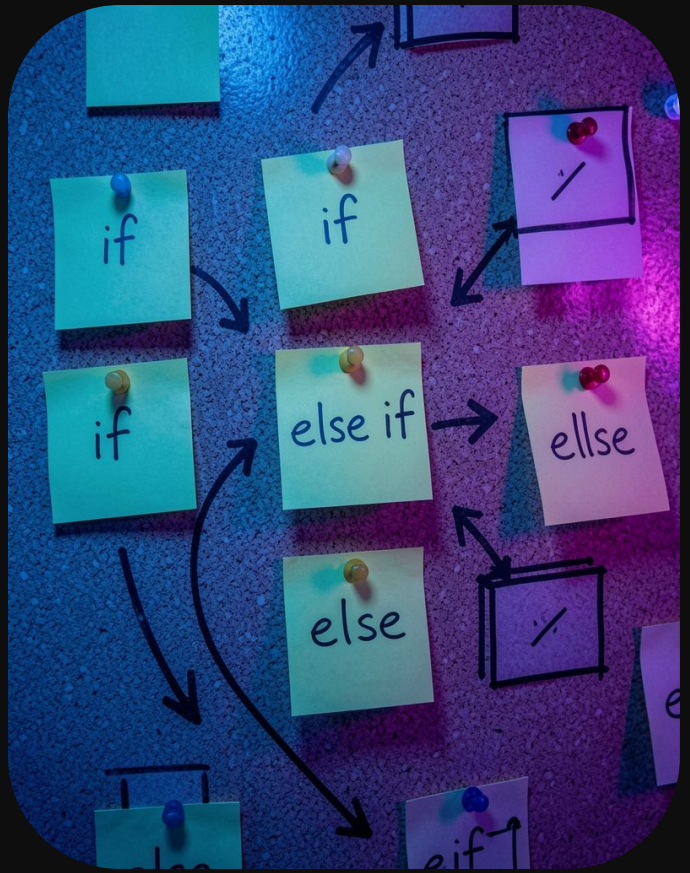
\n\nCombine expressões para controlar fluxo; cuidado com precedência e avaliação preguiçosa (&&, ||).

Operador de Atribuição Composta

Operadores compostos: +=, -=, *=, /=, %= simplificam atualização de variáveis.

\n\nEx.: x += 2; é equivalente a x = x + 2; e mantém tipo original.

04
Estruturas Condicionais





if, else if, else

Estrutura básica para decisões: if (cond) { } else if { } else { }.

\n\nFavor expressões claras e blocos curtos para legibilidade e testes.

Operador ternário

Sintaxe: cond ? valor1 : valor2. Use para atribuições simples.

\n\nPreferir apenas quando a expressão for curta; evite aninhamento complexo.

switch e padrões

switch para múltiplas comparações; *pattern matching* melhora expressividade.

\n\nUse switch para casos discretos e *patterns* para tipos e condições mais ricas.

Conclusões

Reforce tipos, escopo e operadores como base para controle de fluxo em *C#*.

\n\nPratique conversões seguras e escolha estruturas condicionais claras para código robusto.

THANKS!

Do you have any questions?
+55 49 98892-8606



